IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins

UFRN | metrópole DIGITAL

AULA 10

Controle de execução

Introdução ao conceito

- Conceito de execução sequencial.
  - Microcontrolador executa as instruções na ordem em que estão no programa.

- Serve bem a programas simples, mas é um sistema ineficiente quando se trabalha com eventos externos, como pressionar de um botão.



[1]

Controle de execução

- Nesse caso a forma tradicional é o polling (ou varredura).
  - O programa fica em loop verificando constantemente a ocorrência de um evento.
- Tem duas grandes desvantagens:
  - Processador perde tempo verificando algo que pode não ter ocorrido.
  - Como a CPU tem que ficar verificando, não pode entrar em estado de economia de energia.
- Para resolver esses problemas, usa-se interrupções.



[2]

Conceito de interrupção

- Interrupção:
  - Evento assíncrono que altera o fluxo normal do programa.
- Quando ocorre:
  - CPU pausa o que está executando.
  - Salva o estado atual.
  - Desvia a execução para a rotina de tratamento da interrupção.
  - Depois retorna ao ponto em que estava da execução normal.
- Analogia:
  - Contatinho mandando mensagem enquanto estuda para PHIoT.
- Classificada como:
  - Internas – Geradas por periféricos internos do MCU (timer, ADC).
  - Externas – Geradas por pinos de entrada/saída (botão, sensor).

Conceito de interrupção

- Funcionamento:
  - Evento ocorre.
  - CPU interrompe a execução normal e salva o contexto (estado atual dos registrados e PC).
  - MCU consulta o vetor de interrupções, tabela que mapeia cada evento a uma função de tratamento.
  - É executada a ISR (Interrupt Service Routine).
  - Depois, CPU recarrega todos os dados do contexto anterior e continua a execução normal do ponto em que estava.
- Pontos importantes
  - Prioridades – Alguns eventos tem mais prioridade que outros.
  - Mascaramento – Possibilidade de habilitar ou desabilitar certas interrupções.

Boas práticas

- Embora poderosas, deve-se ter cuidado ao utilizar interrupções:
  - Manter a ISR curta – Executar apenas o essencial, pois se longo, pode atrasar o atendimento de outras rotinas.
  - Devem ser leves – Cálculos ou funções lentas (tipo usar delays) prejudicam as respostas.
  - Deve-se utilizar variáveis voláteis – Variáveis que serão lidas tanto dentro da ISR quanto no programa principal devem ser deste tipo para evitar otimizações incorretas do compilador.
  - Priorizar Comunicação via flags – Na ISR essa flag deve ser alterada para que o programa principal saiba quando processar a lógica mais pesada.

INTERRUPÇÃO ESTILO ARDUINO

Exemplo em estilo Arduino

- Função:
  - attachInterrupt(digitalPinToInterrupt(pin), ISR, mode);
    - pin – pino digital que irá gerar a interrupção.
    - ISR – nome da função que será chamada quando a interrupção ocorrer.
    - mode – define a forma que a interrupção deve ser disparada:
      - RISING – quando muda de LOW para HIGH.
      - FALLING – quando muda de HIGH para LOW.
      - CHANGE – quando o sinal muda de qualquer nível.
      - LOW – quando o pino está em nível baixo.
      - HIGH – quando o pino está em nível alto (alguns modelo de MCU).
- A função ISR não possui parâmetros e nem retorno.
- O pin, dependendo do MCU, varia (1, 2, todos).

Exemplo em estilo Arduino

- A função digitalPinToInterrupt(pin) retorna o número da interrupção.
- Como os pinos das interrupções e seus números variam de acordo com o MCU, essa função realiza o mapeamento adequado.
- Assim, trocando o MCU, desde que o pino tenha interrupção atrelada, o número da interrupção será alocado adequadamente.

Exemplo em estilo Arduino

```ino
// Definição dos pinos
const int ledPin = 10;              // LED no Arduino
const int buttonPin = 2;            // Pino que gerará interrupção

// Variável de controle (precisa ser volatile ou seja, variável global)
volatile bool ledState = false;

// Função de interrupção
void toggleLED() {
  ledState = !ledState;             // Alterna o estado do LED
}

void setup() {
  pinMode(ledPin, OUTPUT);          // Define o pino do led como de saída
  pinMode(buttonPin, INPUT_PULLUP); // Botão com resistor interno

  // Configura interrupção
  attachInterrupt(digitalPinToInterrupt(buttonPin), toggleLED, FALLING);
}

void loop() {
  // Atualiza o LED de acordo com a flag modificada pela ISR
  digitalWrite(ledPin, ledState);
}
```

[3]

Necessito de mais...

- Mas o exemplo mostrado ainda não é preciso o suficiente.
- Se o usuário quiser trabalhar com millis() ou micros(), ambas são funções de alto nível que trabalham também com interrupções.
- Nesse caso, o uso de uma interrupção, se lenta, pode afetar os contadores do millis e micros (não incrementariam no tempo certo).
- Então, o que fazer para ter real precisão?
- Resposta: trabalhar direto com o hardware.

Necessito de mais...

- Para maior precisão no tempo de captura pode-se usar o ICP (Input Capture).
- Esse é um hardware de captura do microcontrolador (muitos como o UNO ou ESP32) captura automaticamente o valor de um timer interno no momento que ocorre um evento num pino.
- O usuário pode configurar um timer independente apenas para o calculo de tempo preciso, que independe do timer usado no millis() e micros().
- Necessário habilitar as interrupções manualmente usando a macro sei() (Set Interrupt).

Necessito de mais...

- ISR – Macro definida na biblioteca da AVR que gera a rotina de interrupção associada a um evento.
- TIMER1_CAPT_vect é o nome simbólico do vetor de interrupção associado ao evento Input Capture do Timer1 (ATmega238P).

```
// Configuração do Timer1 e captura de borda
volatile unsigned int lastCapture = 0;
volatile unsigned int periodTicks = 0;
const float prescaler = 8.0;              // mesmo prescaler do Timer1
const float F_CPU_HZ = 16000000;          // clock do Arduino Uno

ISR(TIMER1_CAPT_vect) {
  unsigned int capture = ICR1;            // pega o timestamp
  periodTicks = capture - lastCapture;    // diferença de ticks
  lastCapture = capture;                  // atualiza referência
}

void setup() {
  Serial.begin(9600);
  // Timer1 modo normal
  TCCR1A = 0;
  TCCR1B = (1 << ICES1) | (1 << CS11);    // borda de subida, prescaler = 8
  TIMSK1 = (1 << ICIE1);                  // habilita interrupção de Input Capture
  sei(); // habilita interrupções globais
}

void loop() {
  // Copia periodTicks de forma "segura"
  noInterrupts();
  unsigned int ticks = periodTicks;
  interrupts();
  if (ticks > 0) {
    // Converte ticks para micros
    float time_us = ticks * prescaler * 1000000.0 / F_CPU_HZ;
    Serial.print("Tempo entre bordas: ");
    Serial.print(time_us);
    Serial.println(" us");
    // Se quiser, reseta ticks para evitar repetição
    periodTicks = 0;
  }
  delay(100); // apenas para não poluir o Serial
}
```

[3]

Necessito de mais...

- Registradores:
  - O timer1 tem dois registradores que o controlam:
    - TCCR1A – controla a configuração de saída do timer (modo comparação, PWM nos pinos OC1A e OC1B).
    - TCCR1B – configura clock, prescaler, captura.
      - ICES define a borda.
      - CS10 define o prescaler (frequência de 16MHz).

```
// Configuração do Timer1 e captura de borda
volatile unsigned int lastCapture = 0;
volatile unsigned int periodTicks = 0;
const float prescaler = 8.0;              // mesmo prescaler do Timer1
const float F_CPU_HZ = 16000000;          // clock do Arduino Uno

ISR(TIMER1_CAPT_vect) {
  unsigned int capture = ICR1;            // pega o timestamp
  periodTicks = capture - lastCapture;    // diferença de ticks
  lastCapture = capture;                  // atualiza referência
}

void setup() {
  Serial.begin(9600);
  // Timer1 modo normal
  TCCR1A = 0;
  TCCR1B = (1 << ICES1) | (1 << CS11);    // borda de subida, prescaler = 8
  TIMSK1 = (1 << ICIE1);                  // habilita interrupção de Input Capture
  sei(); // habilita interrupções globais
}

void loop() {
  // Copia periodTicks de forma "segura"
  noInterrupts();
  unsigned int ticks = periodTicks;
  interrupts();
  if (ticks > 0) {
    // Converte ticks para micros
    float time_us = ticks * prescaler * 1000000.0 / F_CPU_HZ;
    Serial.print("Tempo entre bordas: ");
    Serial.print(time_us);
    Serial.println(" us");
    // Se quiser, reseta ticks para evitar repetição
    periodTicks = 0;
  }
  delay(100); // apenas para não poluir o Serial
}
```

[3]

Necessito de mais...

- Registradores:
  - O timer1 tem dois registradores que o controlam:
    - TIMSK1 – Registrador máscara que habilita ou desabilita as fontes de interrupção.
      - ICIE1 – Input Capture Interrupt Enable.
      - TOIE1 – Time Overflow Interrupt Enable.
      - OCIE1B e OCIE1A – Output Compare B/A Match Interrupt Enable.

```
// Configuração do Timer1 e captura de borda
volatile unsigned int lastCapture = 0;
volatile unsigned int periodTicks = 0;
const float prescaler = 8.0;              // mesmo prescaler do Timer1
const float F_CPU_HZ = 16000000;          // clock do Arduino Uno

ISR(TIMER1_CAPT_vect) {
  unsigned int capture = ICR1;            // pega o timestamp
  periodTicks = capture - lastCapture;    // diferença de ticks
  lastCapture = capture;                  // atualiza referência
}

void setup() {
  Serial.begin(9600);
  // Timer1 modo normal
  TCCR1A = 0;
  TCCR1B = (1 << ICES1) | (1 << CS11);    // borda de subida, prescaler = 8
  TIMSK1 = (1 << ICIE1);                  // habilita interrupção de Input Capture
  sei(); // habilita interrupções globais
}

void loop() {
  // Copia periodTicks de forma "segura"
  noInterrupts();
  unsigned int ticks = periodTicks;
  interrupts();
  if (ticks > 0) {
    // Converte ticks para micros
    float time_us = ticks * prescaler * 1000000.0 / F_CPU_HZ;
    Serial.print("Tempo entre bordas: ");
    Serial.print(time_us);
    Serial.println(" us");
    // Se quiser, reseta ticks para evitar repetição
    periodTicks = 0;
  }
  delay(100); // apenas para não poluir o Serial
}
```

[3]

INTERRUPÇÃO ESTILO ESP-IDF

Exemplo estilo ESP-IDF

- Existem algumas funções e configurações associadas à interrupção no estilo ESP-IDF:
  - gpio_config() – Nas configurações do pino, o campo intr_type determina o tipo de trigger de interrupção, tendo como opções:
    - GPIO_INTR_POSEDGE – Borda de subida.
    - GPIO_INTR_NEGEDGE – Borda de descida.
    - GPIO_INTR_ANYEDGE – Qualquer borda.
    - GPIO_INTR_DISABLE – Interrupção desabilitada.
  - gpio_install_isr_service() – Função que instala o serviço global de interrupções do driver da GPIO. Tem como parâmetros intr_alloc_flags, que pode assumir alguns valores (constantes) para definir o funcionamento da interrupção. Essas constantes podem ser combinadas com o uso do operador OR (|).

Exemplo estilo ESP-IDF

- Existem algumas funções e configurações associadas à interrupção no estilo ESP-IDF:
  - gpio_install_isr_service() –
    - Assinatura:
      - esp_err_t gpio_install_isr_service(int intr_alloc_flags)
    - Valores que intr_alloc_flags pode assumir:
      - 0 – Valor padrão, sem flags especiais.
      - ESP_INTR_FLAG_DEFAULT – É frequentemente definido como 0 ou uma combinação de flags padrão. É o valor mais comum em exemplos.
      - ESP_INTR_FLAG_IRAM – Tenta alocar o *handler* de interrupção na IRAM (Instruction RAM). Isso é importante se a rotina de interrupção precisar ser executada quando o cache do Flash estiver desabilitado (o que pode ocorrer em algumas situações).

Exemplo estilo ESP-IDF

- Existem algumas funções e configurações associadas à interrupção no estilo ESP-IDF:
  - gpio_install_isr_service() –
    - Valores que intr_alloc_flags pode assumir:
      - ESP_INTR_FLAG_LEVEL1 – Especifica o nível de prioridade da interrupção podendo assumir valore até ESP_INTR_FLAG_LEVEL5. Para o serviço GPIO ISR, geralmente se usa um nível mais baixo, como ESP_INTR_FLAG_LEVEL1.
      - ESP_INTR_FLAG_EDGE – Aloca a interrupção como sensível à borda.
  - gpio_isr_handler_add() – Função usada para registrar uma rotina de serviço de interrupção (ISR) para um GPIO específico (depois de ter chamado a gpio_install_isr_service()).

Exemplo estilo ESP-IDF

- Existem algumas funções e configurações associadas à interrupção no estilo ESP-IDF:
  - gpio_isr_handler_add() –
    - Assinatura:
      - esp_err_t gpio_isr_handler_add(gpio_num_t gpio_num, gpio_isr_t isr_handler, void *args)
    - Seus parâmetros:
      - gpio_num – tipo gpio_num_t, o número do pino GPIO para o qual você está adicionando o handler de interrupção.
      - isr_handler – tipo gpio_isr_t (que é um ponteiro para uma função do tipo void (*)(void*)) é a função de tratamento da interrupção (ISR) que será executada quando a interrupção no pino gpio_num for acionada.
      - Args – tipo void, é o ponteiro para o argumento que será passado para a função isr_handler quando for executada (geralmente o número do pino).

Exemplo estilo ESP-IDF

- Existem algumas funções e configurações associadas à interrupção no estilo ESP-IDF:
  - ISR – É a função chamada automaticamente quando o evento ocorre.
    - Assinatura:
      - Static void IRAM_ATTR gpio_isr_handler(void *args)
    - Detalhes importantes:
      - IRAM_ATTR – garante que a ISR será alocada na RAM (necessário caso a flash não esteja acessível).
      - A ISR não pode usar printf, malloc, vTaskDelay, etc.
      - Para comunicações externas usa-se funções terminadas com FromISR (como xQueueSendFromISR() ou vTaskNotifyGiveFromISR()).
  - gpio_isr_handler_remove() – remove um handler de um pino (esp_err_t gpio_isr_handler_remove(gpio_num_t gpio_num)).

Exemplo estilo ESP-IDF

- Existem algumas funções e configurações associadas à interrupção no estilo ESP-IDF:
  - gpio_uninstall_isr_service() – Remove o serviço global de interrupção (void gpio_uninstall_isr_service()).
  - xQueueSendFromISR() / vTaskNotifyGiveFromISR() – Formas segura de enviar dados de uma ISR para uma task.
    - Assinatura:
      - BaseType_t xQueueSendFromISR(QueueHandle_t xQueue,const void *pvItemToQueue,BaseType_t *pxHigherPriorityTaskWoken)
    - Seus parâmetros:
      - xQueue – *handle* da fila para a qual o item será enviado.
      - pvItemToQueue – Um ponteiro para os dados que serão copiados para a fila.

Exemplo estilo ESP-IDF

- Existem algumas funções e configurações associadas à interrupção no estilo ESP-IDF:
  - xQueueSendFromISR() / vTaskNotifyGiveFromISR() – Formas segura de enviar dados de uma ISR para uma task.
    - Seus parâmetros:
      - pxHigherPriorityTaskWoken – Um ponteiro para uma variável BaseType_t, que deve ser inicializada como pdFALSE antes de ser passada. A função a definirá como pdTRUE se a operação de envio desbloquear uma tarefa de prioridade mais alta.
    - Assinatura:
      - void vTaskNotifyGiveFromISR(TaskHandle_t xTaskToNotify,BaseType_t *pxHigherPriorityTaskWoken)
    - Seus parâmetros:
      - xTaskToNotify – O *handle* da tarefa que receberá a notificação (e terá seu valor de notificação incrementado).

Fontes

[1] – https://www.tediado.com.br/01/32-situacoes-que-te-fazem-querer-apertar-o-botao-de-reset-na-vida/.
[2] – https://tenor.com/pt-BR/view/acorda-gif-19821809.
[3] – Autoria própria.

IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins
daniel.martins@imd.ufrn.br

UFRN | metrópole DIGITAL